ENTRA ID ACCESS MANAGEMENT

Users, groups and least-privilege RBAC for a hub-and-spoke Azure environment | Version 1

Group-based role assignments scoped to hub, dev and prod, with a joiner, mover and leaver lifecycle, built with Terraform

Project	Entra ID Access Management for a Hub-and-Spoke Azure Environment
Version / date	Version 1, November 6, 2026
Author	Bryan Sykes
Platform	Microsoft Entra ID (free tier) and Azure role-based access control
IaC tool	Terraform 1.16.4; providers azurerm v3.117.1, azuread v2.53.1, random v3.9.1
Remote state	Azure Blob (TerraformStateRG / sykestfstate), own key: entra-access.terraform.tfstate
Companion projects	Personal Azure Infrastructure (hub-and-spoke network); Azure Cost Visibility and Guardrails
GitHub	github.com/MrBSykes/azure-entra-id-access-management

1. The Business Problem

> **Fictional scenario, built to practice a real-world problem**
> A non-profit's Azure environment is run by a rotating mix of staff, volunteers and contractors. Access was granted one person at a time, usually with whatever role made the request go away fastest. Nobody could answer "who can change production?", and when a contractor's engagement ended, nobody was sure that everything they could reach had been removed.

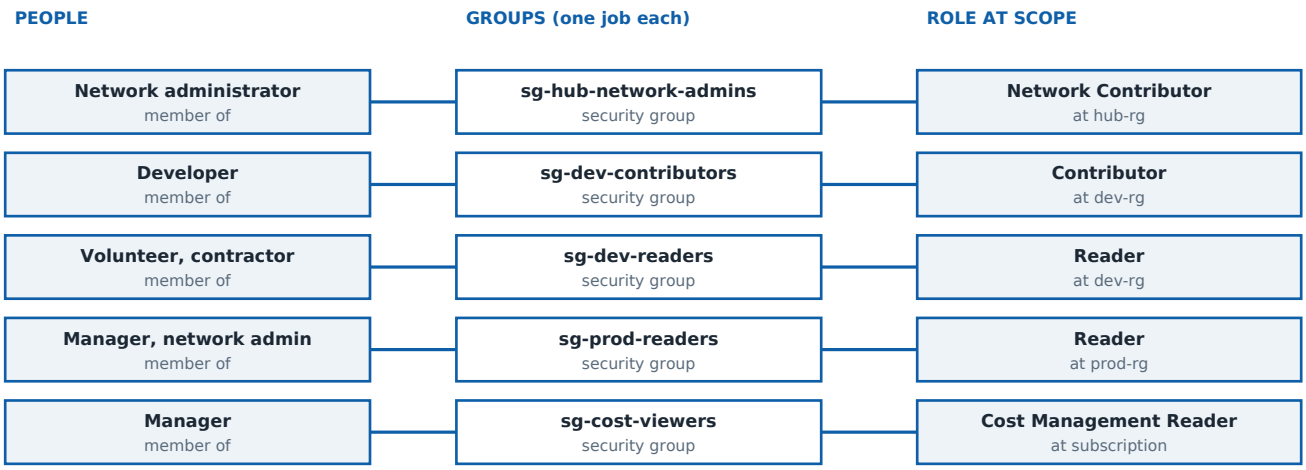
The root problem is access that is tied to people instead of jobs. Per-person grants pile up, cannot be reviewed quickly, and fail silently when someone leaves or changes roles.

Failure	What it caused	What this project adds
Access granted per person	Permissions piled up and nobody could review them	Roles are held by groups; people are members of groups
Over-broad roles	A volunteer could reach production	One built-in role at the smallest resource group that fits
No answer to "who can change prod?"	Production risk was unknown	No group has write access to prod, shown in the portal
Unclear offboarding	Contractors kept access after leaving	A defined leaver path: disable first, then remove

Intended outcome: access follows the job. A joiner, a role change and a leaver are each a small, reviewable change with a plan that shows exactly what will happen before it happens.

2. The Access Model

People are never given roles directly. Each group represents one job in one environment and holds exactly one built-in role at one scope. Reading a group name tells you what its members can do and where.



Resource	Terraform type	Purpose
5 lab users	azuread_user (for_each)	Fictional accounts: manager, developer, network admin, volunteer, contractor
5 initial passwords	random_password (for_each)	Generated, never printed; users must change them at first sign-in
5 security groups	azuread_group (for_each)	One per job function and environment
8 group memberships	azuread_group_member (for_each)	Link each person to their groups
5 role assignments	azurerm_role_assignment (for_each)	One built-in role per group at one scope
3 resource group lookups	data azurerm_resource_group	Read-only references to hub-rg, dev-rg and prod-rg

Nobody has write access to prod
No group is granted a write role on prod-rg. Production changes are meant to flow through reviewed code, which is the reason for the CI/CD item on my roadmap.

3. Component Deep Dives

Each component answers four questions: what it does, what business problem it solves, how it was built, and what I would do differently.

3.1 Lab users

What it does	Creates cloud-only accounts in the Entra ID tenant with a job title, department and employee type, so each account looks like a real role.
Business problem	Access decisions only make sense for realistic roles. A manager, a developer, a volunteer and a contractor each need different access.
How it was built	Users are defined as data in lab.auto.tfvars and created with for_each. Each gets a generated 20-character password and must change it at first sign-in. A lifecycle rule ignores later password changes so Terraform does not fight a user who has reset theirs.
Next time	Use a verified company domain, and have users register their own credentials instead of storing initial passwords in state.

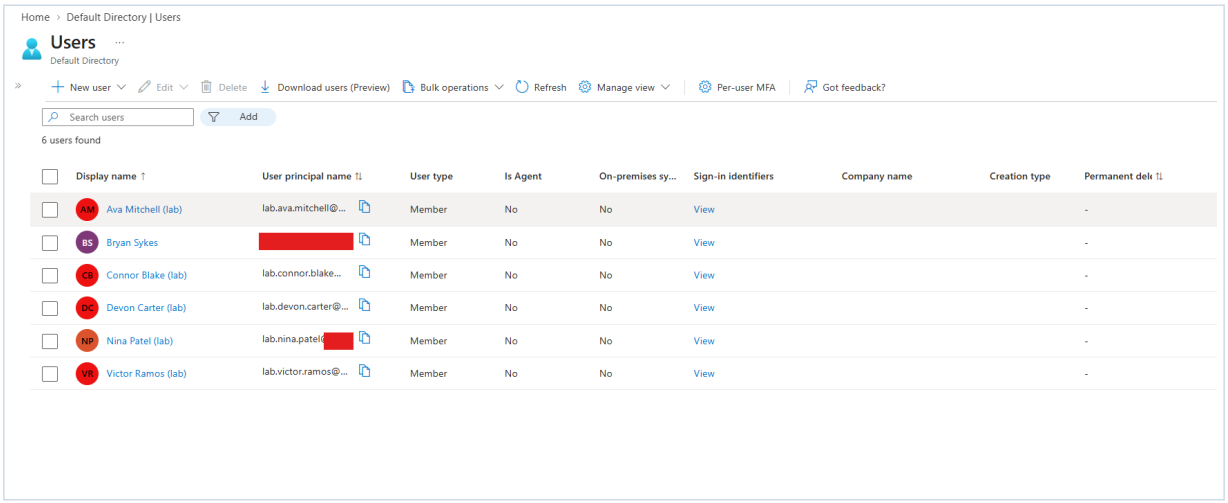


Figure 1. The five lab users, marked (lab), plus my admin account: six users in the tenant. (screenshots/01-entra-users-list.png)

3.2 Security groups and memberships

What it does	A security group collects users so permissions can be granted once and inherited by every member.
Business problem	Granting roles to individuals does not scale and cannot be reviewed. A group per job makes review a short list.
How it was built	Five assigned-membership security groups, one per job and environment, plus eight memberships generated from each user's group list. A precondition fails the plan with a clear message if a user names a group that does not exist.
Next time	Add owners to every group and, with the right licenses, dynamic membership rules so department changes move people automatically.

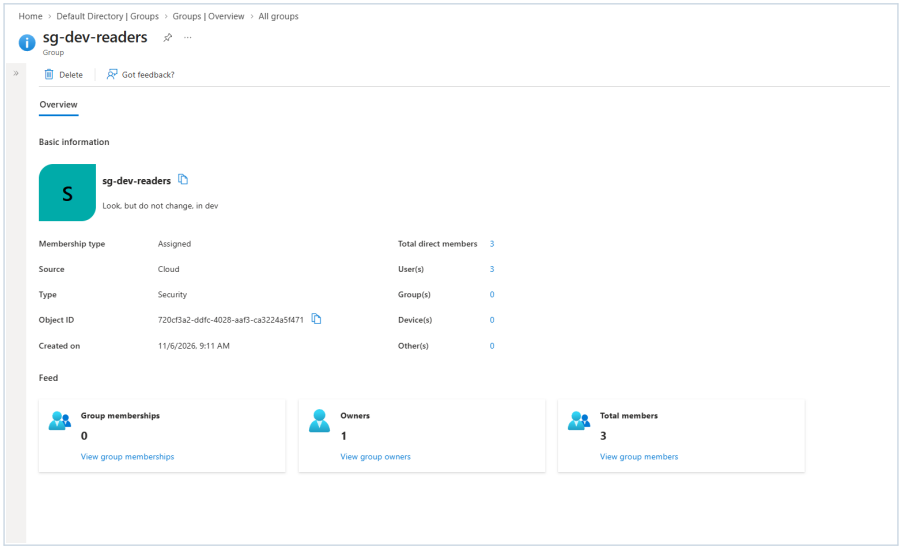


Figure 2. sg-dev-readers: a Security group with Assigned membership and its members. (screenshots/02-group-members-sg-dev-readers.png)

3.3 Role assignments

What it does	Azure RBAC grants a role to a security principal at a scope. Roles inherit downward: a role on a resource group applies to everything in it.
Business problem	A contractor who needs dev should not be able to see prod. Scope is what makes the boundary real.
How it was built	Each group gets one built-in role at one scope (resource group, or the subscription only for cost viewing). Assignments set principal_type to Group, which avoids a replication-delay error for newly created principals, and a precondition checks the scope exists.
Next time	Add a custom role for one narrow task, and review assignments on a schedule.

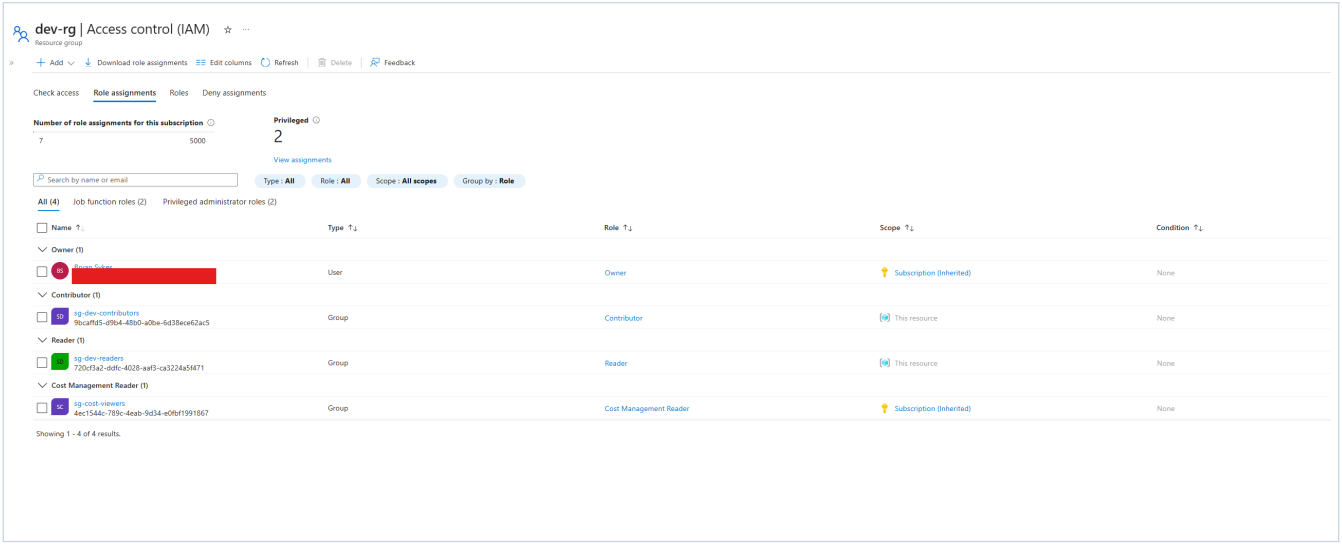


Figure 3. dev-rg: Contributor for sg-dev-contributors, Reader for sg-dev-readers, and the inherited cost role. (screenshots/03-dev-rg-role-assignments.png)

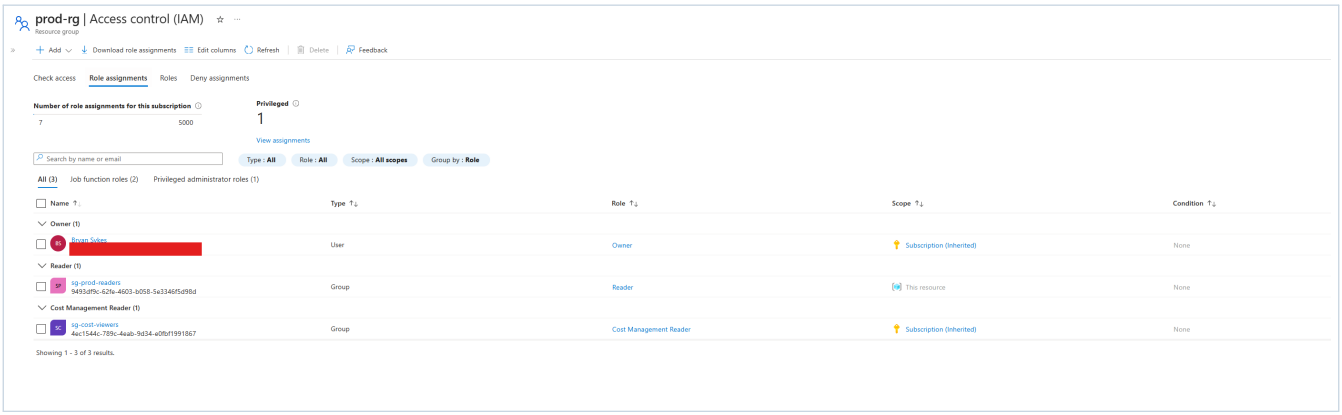


Figure 4. prod-rg: only Reader (sg-prod-readers) and the inherited cost role. No group has a write role here. (screenshots/04-prod-rg-role-assignments-no-write.png)

4. Lifecycle Walkthrough: Joiner, Mover, Leaver

Each lifecycle event is a change to the data file and a Terraform plan that shows the exact effect before anything is applied. A small PowerShell script runs each step: it edits the data file, shows the plan and applies only if I type a lowercase y. After every step, a plain plan reported no changes, so the code and the tenant agreed.

Event	Change	Plan result	Evidence
Joiner: five users start	Initial apply	28 added: 5 users, 5 passwords, 5 groups, 8 memberships, 5 role assignments	Figure 1
Mover: a volunteer becomes a developer	Victor moved from sg-dev-readers to sg-dev-contributors	1 to add, 0 to change, 1 to destroy (one membership replaced)	Plan output; group Members page
Leaver, step 1: contractor engagement ends	Connor set to account_enabled = false	0 to add, 1 to change, 0 to destroy (account_enabled true to false)	Plan output
Leaver, step 2: after retention	Connor's entry removed from the data file	Account removed from the tenant	Figures 5 and 6

Why disable before delete. Disabling stops sign-in immediately and is trivially reversible. The disabled account still holds its group memberships, so its access is dormant, not removed. The second step removes the account and its membership entirely. Entra then keeps the deleted user recoverable for 30 days.

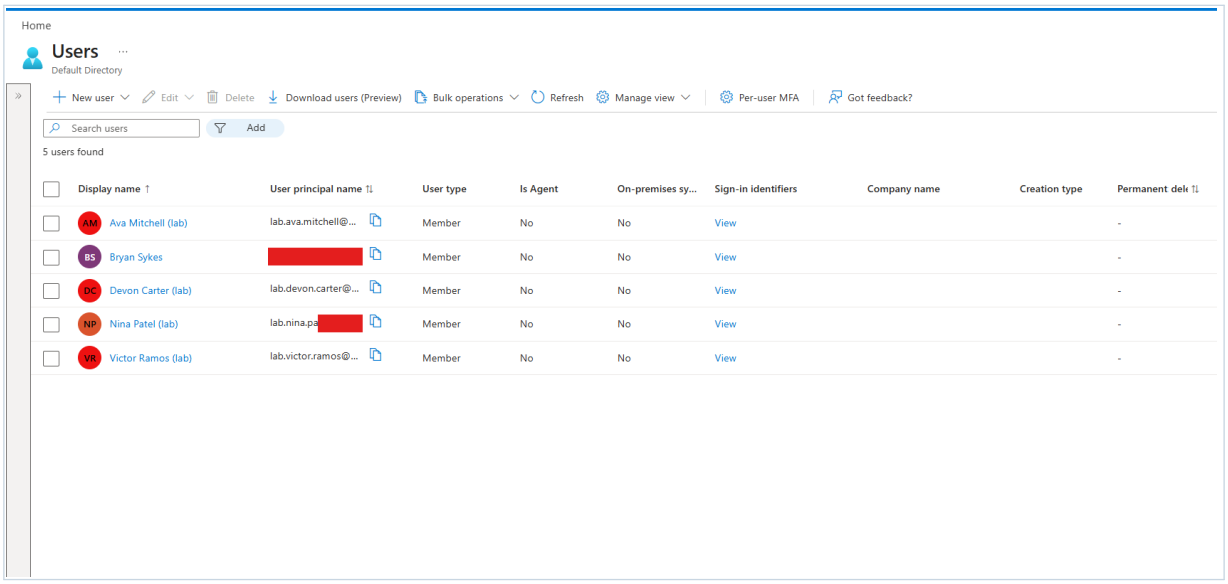


Figure 5. After the leaver step the tenant has five users. The contractor is gone from the Users list. (screenshots/05-users-after-leaver-connor-removed.png)

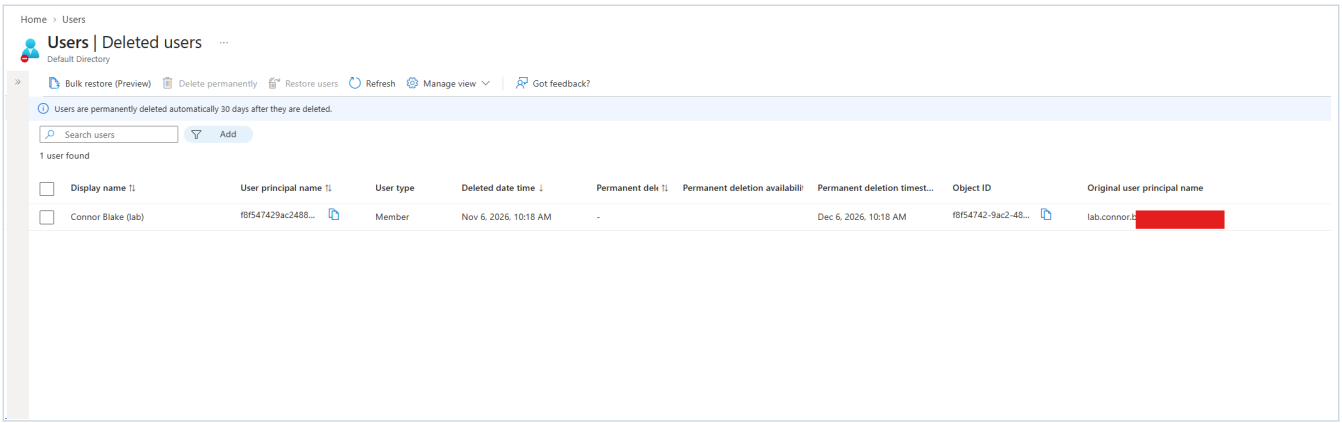


Figure 6. Deleted users: the removed contractor, with permanent deletion automatic after 30 days (December 6, 2026). (screenshots/06-deleted-users-connor-recoverable.png)

A gap I am being honest about
I ran the disable and delete steps back to back and did not test a sign-in as the disabled user, so I have no "account disabled" screenshot. The plan output and the Deleted users page are the evidence. Next time I would test the sign-in for each lifecycle state before moving to the next one.

5. Least-Privilege Validation

To prove the model works and does not just look right in code, I signed in as the developer (Devon Carter, member of sg-dev-contributors, Contributor on dev-rg) in a private browser window, which kept my admin session separate. Security Defaults was on in the tenant, so Entra required Devon to register an authenticator app before he could continue. I left that control on.

Test	Expected	Result	Fig.
Resource group list as Devon	Only dev-rg visible	Only dev-rg listed	7
Create a resource group as Devon	Denied (no subscription-level write)	"You do not have permissions to create resource groups under subscription"	8
Start a storage account inside dev-rg as Devon	Allowed (Contributor on dev-rg)	Reached Review + create with Create enabled; I did not create it	9
Check access on prod-rg for Devon (admin view)	No assignments	0 role assignments, 0 deny assignments	10

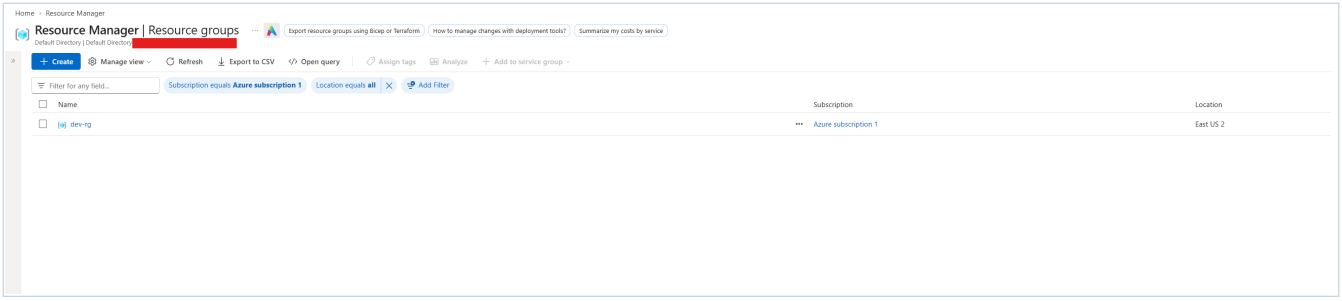


Figure 7. Signed in as the developer: dev-rg is the only resource group visible. (screenshots/07-devon-sees-only-dev-rg.png)

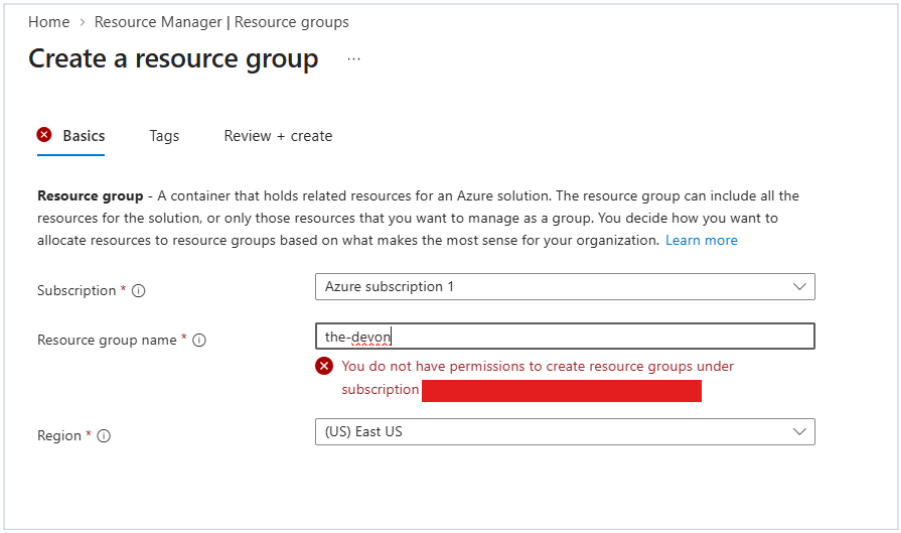


Figure 8. The developer is denied creating a resource group at subscription level. (screenshots/08-devon-denied-create-permission.png)

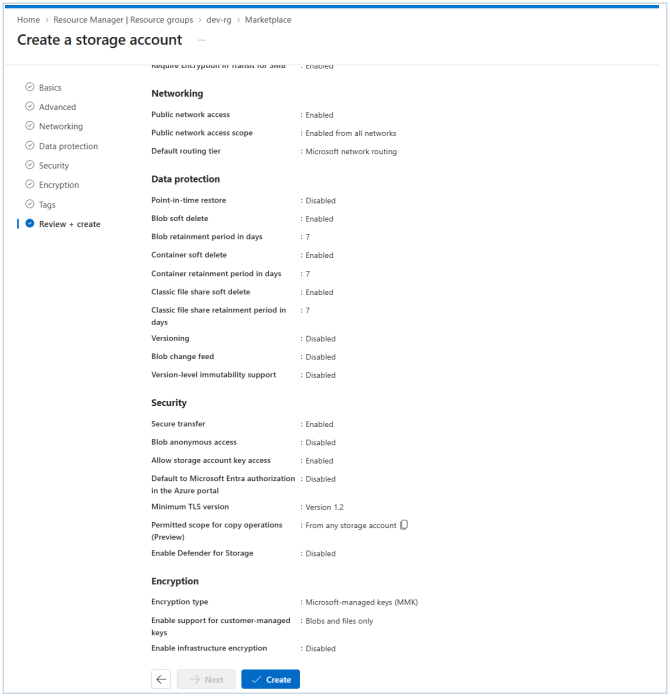


Figure 9. The developer reaches Review + create for a storage account in dev-rg. Create was not clicked. (screenshots/09-devon-can-create-in-dev-rg.png)

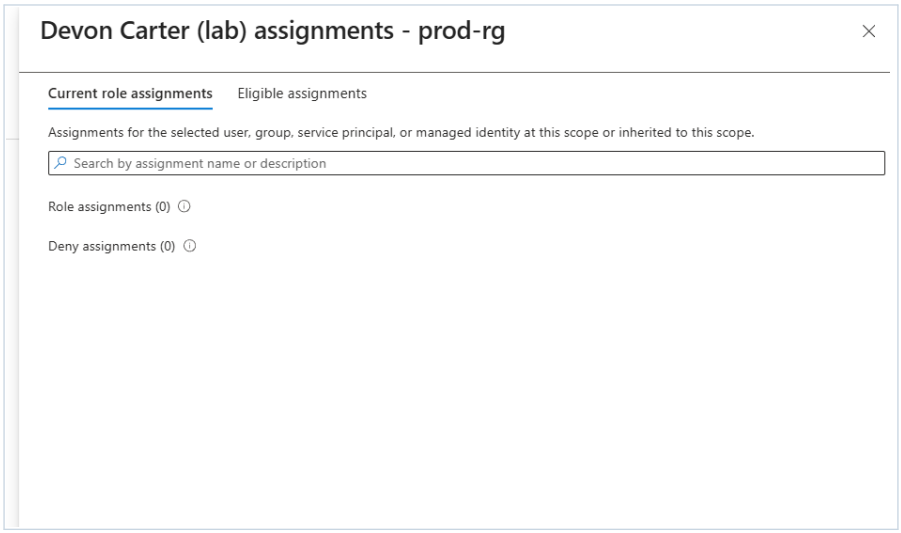


Figure 10. Check access on prod-rg for the developer: 0 role assignments and 0 deny assignments. (screenshots/10-devon-check-access-prod-rg-zero.png)

The create-resource-group denial is a subscription-level action, so on its own it shows scoping rather than the dev-versus-prod boundary. The dev-rg result and the zero assignments on prod-rg are the direct evidence for that boundary.

6. Key Architectural Decisions

Decision	Reasoning	Trade-off
Roles go to groups, never to individuals	Offboarding removes one membership instead of hunting for grants	One-off grants to a person defeat the model
One role at one scope per group	A group name states exactly what it can do and where	More groups than one per team
Scope to resource groups, not the subscription	A contractor with dev access cannot see prod	Subscription roles only where the job needs them (cost viewing)
No write access to prod	Production changes should be reviewed, not made by hand	Slower emergency changes; a real team needs a break-glass process
Users and groups as data	Joiner, mover and leaver are small edits with a readable plan	Portal edits to managed accounts show up as drift
Disable first, delete later	Reversible, and sign-in stops at once	An extra step; memberships remain until removal
Passwords generated, never printed	No shared or guessable credentials; forced change at first sign-in	Passwords live in the state blob, so state access must be restricted
Built-in roles only	Well understood and auditable	Less precise than a custom role in some cases
Free Entra features only	Keeps the lab at no cost	No Conditional Access, dynamic groups, group licensing or access reviews
Lifecycle steps in a script with a confirm prompt	A mistyped answer cannot apply a change	Regex edits are brittle if the file layout changes

7. Troubleshooting Log

#	Symptom	Root cause	Resolution	Lesson
1	My plan summary said 23 resources; the real plan said 28	I miscounted: 5 users + 5 passwords + 5 groups + 8 memberships + 5 role assignments = 28	Counted from the code and checked against the real plan	Do arithmetic from the source, and trust the plan over a summary
2	"Running scripts is disabled on this system"	PowerShell's default execution policy blocks local scripts	Bypass for that window only: Set-ExecutionPolicy -Scope Process	Scope a policy change to the session, not the machine
3	git diff printed usage text inside the script	The project was not yet a git repository	git init, then commit before the next lifecycle step	Put work under version control before changing it
4	Typing "yes" at the apply prompt did nothing	The script accepts only a lowercase y by design	Applied the saved plan file directly	A saved plan applies exactly what was reviewed
5	A lab user could not skip MFA registration	Security Defaults requires an authenticator app	Registered a test authenticator, removed it afterward	Do not disable a security control to finish a lab
6	"The resource ID is invalid" in my first prod test	My URL contained a literal placeholder; not a permissions error	Dropped that attempt; proved the prod boundary with Check access	Make sure an error means what you think before using it as evidence

8. Cost, Limitations and Control Alignment

Users, groups and Azure role assignments carry no charge on the free tier as far as I know, and I have not seen a charge for them in this subscription. The pricing claim is marked as unverified in the README.

Honest limits of this design:

- **Personal tenant.** The lab users exist only in my tenant. A real organization would use a verified company domain and often sync from on-premises Active Directory.
- **No Conditional Access or access reviews.** Those need paid tiers and are the natural next layer.
- **Passwords in state.** A real deployment would use a secrets store or have users register their own credentials.
- **No disabled-sign-in evidence.** See the gap noted in section 4.
- **Drift is not demonstrated yet.** A portal edit to a managed group is a planned follow-up.

> **Control alignment, concepts only**
> This is a personal commercial subscription. It is **not** a FedRAMP or NIST 800-53 compliant environment, and I make no compliance claim. The lab practices the ideas behind AC-2 (account management: create, change, disable and remove through a reviewable change), AC-6 (least privilege: one role at the smallest scope, no write access to prod) and role-based access (permissions attach to groups that represent jobs).

9. Business and Career Alignment

Skill	Evidence in this project
Identity and access management	Users, security groups, group-based RBAC, and a joiner, mover and leaver lifecycle
Least privilege	One role per group at the smallest scope; prod has no write role; verified by signing in as a user
Infrastructure as Code	28 resources defined in Terraform with for_each over data, preconditions, a separate state key, and a plan before every change
Operational habits	Disable before delete, a recoverable offboarding, drift-aware design
Secure engineering habits	Passwords generated and never printed; tenant domain and subscription ID kept out of Git; screenshots redacted
Verification and honesty	Corrected my own miscount, dropped a screenshot that proved nothing, and documented the gap I missed

User and access administration is a core duty of entry-level cloud and IT roles. This project supports the identity and governance objectives of the Azure Administrator (AZ-104) path, which is my next certification goal, alongside AZ-900 and CompTIA Security+ already completed.

10. What I Would Do Differently Next Time

- Define the groups and roles before creating any resources, so nobody is ever granted access one person at a time.
- Start from a verified company domain and a break-glass account with strong protection.
- Add Conditional Access and periodic access reviews once the tenant has the licenses for them.
- Use custom roles only where a built-in role is too broad, and review them regularly.
- Keep identity in its own repo and state, as here, so an access change is never bundled with a network change.
- Test the sign-in for each lifecycle state before moving to the next one.

Next steps

Action	Purpose
Add a custom role for a narrow task	Show least privilege beyond built-in roles
Connect pipeline access to a workload identity	Let CI/CD deploy without a user account
Demonstrate drift: change a membership in the portal and show the plan	Show why identity belongs in code
CI that runs fmt, validate and plan on every change	Review every access change before it deploys

11. Conclusion

This project closes the gap behind the fictional non-profit's problem: access that followed people instead of jobs. With five groups, five role assignments and a short script, a joiner, a role change and a leaver are each a small, reviewable change, and the portal confirms what the code says.

Signing in as a lab user turned a design into evidence: the developer could build in dev, could not see or change prod, and the offboarded contractor sits in Deleted users, recoverable for 30 days. The detours along the way, a miscount, a blocked script and a test that proved nothing, are the everyday work of infrastructure engineering.

Documentation prepared by Bryan Sykes, November 6, 2026.